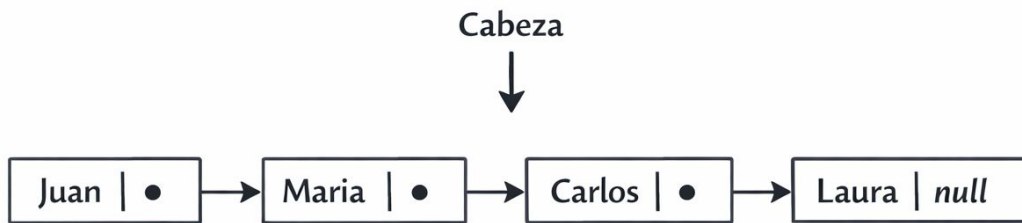


Listas Enlazadas

Estructura de datos

Juan Diego castro



- **Cabeza:** Es el primer nodo de la lista, en este caso "Juan".
- **Nodos:** Cada estudiante representa un nodo con un dato y un **enlace** al siguiente.
- **Enlaces:** Son las flechas que conectan cada nodo.
- **Final de la lista:** El último nodo ("Laura") apunta a **null**, indicando que no hay más elementos.

Descripción del esquema

La lista enlazada tiene cuatro nodos con los nombres Juan, María, Carlos y Laura. Cada nodo guarda un dato y un enlace al siguiente. El primer nodo es la cabeza y desde ahí empieza la lista. Todos los nodos están conectados y el último apunta a null, lo que indica que la lista termina.

Explicación del concepto dinámico

La lista enlazada es dinámica porque no tiene un tamaño fijo desde el inicio. Se pueden agregar o quitar elementos sin tener que reorganizar todo, ya que los nodos se conectan entre sí. Esto la hace útil cuando la cantidad de datos puede cambiar.

Explicación del código

En el código se crea una clase Nodo que guarda un dato y una referencia. Luego se crean varios nodos con nombres y se enlazan entre sí. Después se recorre la lista con un ciclo while para mostrar los datos. También se cuenta cuántos nodos hay en total.

Conclusión

Las listas enlazadas son útiles porque permiten manejar datos de forma flexible. Aunque son más complejas que los arreglos, sirven mucho cuando los datos cambian constantemente.

```
public class ListaEnlazada {  
    // Clase Nodo (cada nodo tiene un dato y una referencia)  
    static class Nodo {  
        String dato;  
        Nodo siguiente;  
        public Nodo(String dato) {  
            this.dato = dato;  
            this.siguiente = null;  
        }  
    }  
    public static void main(String[] args) {  
        Nodo n1 = new Nodo("Juan");  
        Nodo n2 = new Nodo("Maria");  
        Nodo n3 = new Nodo("Carlos");  
        Nodo n4 = new Nodo("Laura");  
  
        // Enlazar nodos  
        n1.siguiente = n2;  
        n2.siguiente = n3;  
        n3.siguiente = n4;  
  
        // Definir la cabeza de la lista  
        Nodo cabeza = n1;  
  
        // Recorrer la lista  
        Nodo actual = cabeza;  
  
        System.out.println("Lista de estudiantes:");
```

```
while (actual != null) {  
    System.out.println("- " + actual.dato);  
    actual = actual.siguiente;  
}  
  
// Contar nodos  
int contador = 0;  
Nodo temp = cabeza;  
  
while (temp != null) {  
    contador++;  
    temp = temp.siguiente;  
}  
  
System.out.println("Total de estudiantes: " + contador);  
}  
}
```